

Informe Técnico: Suite de Skills de Anthropic

Proyecto: Agente Investigador Académico (v7)

Para: Equipo de Desarrollo de Skills de Anthropic

De: [Tu Nombre/Nombre del Proyecto]

Fecha: 5 de noviembre de 2025

Estado: ESPECIFICACIÓN DE REQUISITOS (Versión Piloto)

1.0 Visión General del Proyecto

1.1. Objetivo Principal

El objetivo de este proyecto es desarrollar una **"Suite de Skills" (Skill Pack)** de nivel de postgrado para Claude. Esta suite transformará a Claude en un asistente experto ("Agente Investigador") capaz de guiar a un usuario a través del ciclo completo de investigación y redacción académica, con un enfoque específico en **Humanidades, Ciencias Sociales y Filosofía**.

A diferencia de los asistentes de IA genéricos, este sistema está diseñado para mitigar activamente la alucinación y el plagio. Para lograrlo, implementa una arquitectura multiagente metacognitiva donde cada paso de la investigación es **Generado**, **Criticado** y **Verificado** internamente antes de ser presentado al usuario.

1.2. Principio de Diseño Central: "Cero Brechas para Alucinación"

El pilar fundamental de esta arquitectura es el **rigor metodológico y la integridad académica**. El sistema operará bajo tres principios no negociables:

- Metacognición Externalizada:** El sistema simula un "pensamiento crítico" al estructurar el razonamiento como un diálogo interno entre agentes **Generadores** (creativos) y agentes **Críticos** (escépticos). Ningún **output** se considera "Verificado" hasta que pasa una auditoría crítica.
- Base de Conocimiento Explícita:** El sistema **no** inventa metodologías. El "cómo" de cada tarea (cómo redactar un párrafo, cómo citar, cómo formular una hipótesis) se basa **estrictamente** en una batería de conocimiento de **frameworks** explícitos.
- Corpus Cerrado:** El análisis de contenido se realiza **exclusivamente** sobre el corpus de documentos (PDFs, `.txt`, `.csv`, `.md`, transcripciones de entrevistas, notas de campo) que el usuario proporciona. El sistema no tiene permitido "buscar en la web" para sus análisis, eliminando así la variabilidad y la contaminación de fuentes no verificadas.

1.3. Fundamentos Teóricos y Fuentes de Inspiración

Este diseño se basa en la síntesis de tres artículos teóricos clave y una base de conocimiento práctica proporcionada.

1. **Inspiración Arquitectónica (Multiagente):** El **"The Denario project"** (<https://arxiv.org/abs/2510.26887v1>) inspira nuestra arquitectura de un agente **CONTROL** (orquestador) que gestiona un equipo de subagentes especializados con roles opuestos (ej. **Idea Maker** vs. **Idea Hater**).

2. **Inspiración de Memoria (Metacognición):** El **"Context Engineering 2.0"** (<https://arxiv.org/abs/2510.26493v1>) proporciona el modelo para nuestra memoria persistente (**Research_Log.md**) a través del concepto de **"Self-Baking"** (auto-abstracción) y **"Context Isolation"** (aislamiento de contexto).

3. **Inspiración de Tarea (Análisis):** El **"Inverse Knowledge Search"** (<https://arxiv.org/abs/2510.26854v1>) inspira el **framework** de nuestro agente de revisión de literatura, el **Agente_Socrates**, que busca la **"materia oscura"** (brechas de razonamiento) en el corpus del usuario.

4. **Base de Conocimiento (Algoritmos de Ejecución):**

- Repositorio: <https://github.com/mroaromero/Skills-para-Escritura-y-Razonamiento-LLM>

- Frameworks Centrales:

- 05_apa-7-citacion-rigurosa.md**

- 07_plantilla-tesis.md**

- 09_construccion-parrafos-academicos.md**

- 10_hipotesis-y-recursos-retoricos.md**

2.0 Arquitectura del Sistema (Implementación en Claude)

Este proyecto se implementará como un **"Skill Pack"** de Claude. Esta arquitectura modular es esencial para el **"Aislamiento de Contexto"** y la gestión de la metacognición.

2.1. El **SKILL.md** Principal (Agente **CONTROL** / Orquestador)

El **SKILL.md** principal (el punto de entrada de la Skill) definirá al **Agente CONTROL**. Este agente es el **"Director de Tesis"** y el único con el que el usuario interactúa directamente.

- Función:** Gestionar el **Research_Log.md** (memoria) y orquestar el **"Workflow de 12 Pasos"** (ver Sección 4.0).

- Lógica:** Su **prompt** de sistema le instruirá para que **no ejecute tareas directamente**. En su lugar, debe seguir el **"Algoritmo de Bucle Metacognitivo"** (Sección 3.2) e invocar a los subagentes (otras **skills** o **prompts** definidos) según sea necesario.

2.2. Los Subagentes (Frameworks de Razonamiento Explícitos)

Cada "subagente" es, en efecto, un **prompt** de sistema especializado que el **CONTROL** invoca. Estos **prompts** se almacenarán en la carpeta **references/** de la Skill. El **Agente_CONTROL** "cargará" estos **frameworks** para adoptar la personalidad y el algoritmo del subagente requerido.

****Archivos de Frameworks a Implementar (`references/`):****

1. **``framework_socrates.md`:`** (Basado en "Inverse Knowledge Search") Define la lógica para el análisis de la literatura (Paso 2).
2. **``framework_hipotesis.md`:`** (Basado en `10\_hipotesis-y-recursos-retoricos.md`) Define la lógica para la formulación de hipótesis (Paso 6).
3. **``framework_analisis.md`:`** Define la lógica para la bifurcación metodológica (Paso 7) y los agentes de análisis (Paso 10).
4. **``framework_redaccion.md`:`** (Basado en `07\_plantilla-tesis.md` y `09\_construccion-parrafos-academicos.md`) Define la lógica del `Agente\_Plato` (Paso 11).
5. **``framework_auditoria.md`:`** (Basado en `05\_apa-7-citacion-rigurosa.md`) Define la lógica del `Agente\_REVIEWER` (Paso 12).
6. **``framework_critico_generico.md`:`** Define la lógica del `Agente\_Hater` (Crítico) para los pasos de planificación.
7. **``knowledge_base/`:`** Una carpeta que contendrá *toda* la batería de conocimiento del repositorio `Skills-para-Escritura-y-Razonamiento-LLM` (ej. `Fallacies.txt`, `Arguments.txt`, etc.).

2.3. Gestión de Memoria y Estado: El `Research\_Log.md`

Este es el componente técnico más crítico para prevenir la deriva de contexto y la alucinación.

* ****Archivo de Estado:**** El sistema debe crear y mantener un archivo `Research\_Log.md` por cada proyecto.

* ****Función ("Self-Baking"):** Este archivo es la memoria persistente. No es una transcripción del chat. Es un ****documento de estado estructurado**** que contiene ***solo*** los ****outputs verificados**** (aprobados por los `Críticos`) de cada paso del workflow.

* ****Lógica de Escritura:**** El `Agente\_CONTROL` es el ***único*** agente que puede escribir en este archivo. Al final de cada paso exitoso, el `CONTROL` "auto-abstrae" el resultado y lo añade al log.

* ****Lógica de Lectura:**** Cuando un agente es invocado (ej. `Agente\_Plato` en Paso 11), ****no**** recibe el historial de chat. Recibe ***únicamente*** el `Research\_Log.md` como contexto. Esto asegura que solo redacte sobre hechos, hipótesis y análisis ***verificados***.

****Ejemplo de Estructura del `Research\_Log.md`:****

`` `markdown

Research_Log: [Nombre del Proyecto]

ESTADO: Activo

FASE ACTUAL: 2. Metodología

PASO ACTUAL: 7. Metodología de la Investigación

PASO 1: Selección del Tema

* **Status:** Verificado

* **Output:** "Análisis del discurso de polarización en líderes de opinión de Twitter (X) durante el último año electoral."

PASO 2: Revisión de Literatura (Base: Corpus del Usuario)

* **Status:** Verificado

* **Brecha Identificada (Materia Oscura):** "La literatura existente es 90% cuantitativa (frecuencia de hashtags). La 'brecha' es la falta de un análisis retórico-cualitativo de los argumentos."

* **Fuentes Clave (del Corpus):** `[fuente\_usuario\_1.pdf]`, `[fuente\_usuario\_3.pdf]`

PASO 6: Hipótesis

* **Status:** Verificado

* **Hipótesis (Cualitativa):** "Se asume que los líderes de opinión utilizan predominantemente falacias de 'Hombre de Paja' y 'Apelación a la Emoción' para enmarcar a los adversarios."

* **Clasificación:** Supuesto de Trabajo (Cualitativo).

3.0 Lógica Central de Orquestación (Agente CONTROL)

Este es el algoritmo de metacognición principal del sistema. El Agente CONTROL (el SKILL.md principal) debe ejecutar este bucle para cada uno de los 12 pasos del workflow.

3.1. Algoritmo de Bucle Metacognitivo (Generar-Criticar)

1. **INICIAR PASO:** El CONTROL identifica el paso actual (ej. "Paso 6: Hipótesis").
 2. **ACTUALIZAR LOG (RECITAR):** El CONTROL escribe en el Research_Log: PASO ACTUAL: 6. Formulación de Hipótesis. STATUS: En Progreso. (Esto actúa como un "recitado" de objetivos para mantener el enfoque).
 3. **INVOCAR GENERADOR:** El CONTROL "carga" el framework del agente generador relevante (ej. framework_hipotesis.md) y le pide que ejecute su tarea.
 4. **RECIBIR OUTPUT BORRADOR:** El CONTROL recibe el output_borrador del Generador.
 5. **INVOCAR CRÍTICO:** El CONTROL "carga" el framework del agente crítico (framework_critico_generico.md o framework_auditoria.md) y le entrega el output_borrador para su evaluación.
 6. **RECIBIR CRÍTICA:** El CONTROL recibe un informe_de_critica estructurado (JSON: {veredicto: "APROBADO" | "RECHAZADO", motivos: [...]}).
 7. **DECIDIR (Metacognición):**
 - **CASO A: Veredicto == "APROBADO"**
 - CONTROL escribe el output_borrador en el Research_Log con STATUS: Verificado.
 - CONTROL avanza al Paso [X+1].
 - **CASO B: Veredicto == "RECHAZADO" (Iteración 1-2)**
 - CONTROL vuelve al Paso 3, pero esta vez la invocación incluye la crítica.
 - *Prompt de CONTROL:* "Invocando Agente_Hipotesis (Intento 2). Tarea: Reformular. Borrador anterior: [...]. Crítica del Hater: [...]."
 - **CASO C: Veredicto == "RECHAZADO" (Iteración 3)**
 - CONTROL se detiene.
 - *Prompt de CONTROL:* "Fracaso de la metacognición. El Agente Crítico ha rechazado 3 veces el trabajo. La última crítica es: [...]. Necesito tu intervención."
-

4.0 El Workflow de 12 Pasos (Lógica Detallada)

El Agente_CONTROL debe guiar al usuario a través de este ciclo.

FASE 1: PLANIFICACIÓN (QUÉ y CUÁL ES LA BASE)

- **Paso 1-5: Del Tema a la Justificación**
 - **Agentes Invocados:** Agente_Generador_Ideas vs. Agente_Critico_Generico.
 - **Lógica:** El CONTROL ejecuta el bucle Generar-Criticar para los Pasos 1, 3, 4 y 5.
 - **Agente_Socrates (Paso 2):** Es un Generador especial. Su framework (framework_socrates.md) le instruye a leer el corpus del usuario y aplicar la "Búsqueda Inversa" [3] para definir la "Brecha de Conocimiento".
- **Paso 6: Marco Teórico e Hipótesis**
 - **Agentes Invocados:** Agente_Hipotesis (Generador) vs. Agente_Critico_Hipotesis (Hater).
 - **Lógica:** El Generador debe ejecutar el algoritmo de la Sección 5.1. El Crítico debe auditar usando el mismo checklist.

FASE 2: METODOLOGÍA (CÓMO)

- **Paso 7: Metodología (La Bifurcación)**
 - **Agentes Invocados:** Agente_Metodologia (Generador).
 - **Lógica:** Este agente es crucial. Lee la Hipótesis (Paso 6) y decide la ruta:
 - Si la hipótesis es Causal/Correlacional (cuantitativa), propone la ruta ENGINEER.
 - Si es un Supuesto de Trabajo (cualitativo), propone la ruta HERMENEUTA.
 - El CONTROL solicita la validación del usuario para esta ruta.
- **Pasos 8-9: Muestra e Instrumentos**
 - **Lógica:** El Generador define el corpus (ej. "los 50 PDFs del usuario") o la muestra (ej. "el datos.csv del usuario"). El Crítico valida que esta muestra sea suficiente para probar la hipótesis del Paso 6.

FASE 3: EJECUCIÓN Y REPORTE (EL INFORME)

- **Paso 10: Análisis de Datos (Ejecución de la Bifurcación)**
 - **Agentes Invocados:** Agente_HERMENEUTA o Agente_ENGINEER.
 - **Lógica HERMENEUTA:** Ejecuta el algoritmo de la Sección 5.2.
 - **Lógica ENGINEER:** Ejecuta el algoritmo de la Sección 5.3.
 - El CONTROL recibe los *outputs* (un informe temático o tablas/gráficos) y los guarda en el Research_Log como "Hallazgos Verificados".
- **Paso 11: Redacción del Borrador (Tesis/Paper)**
 - **Agentes Invocados:** Agente_Plato (Generador).
 - **Lógica:** El Agente_Plato es invocado. Se le prohíbe el acceso al historial de chat. Se le entrega *únicamente* el Research_Log.md y el corpus del usuario. Ejecuta el algoritmo de la Sección 5.4.
- **Paso 12: Auditoría de Integridad y Conclusión**
 - **Agentes Invocados:** Agente_REVIEWER (Crítico Final).
 - **Lógica:** El REVIEWER es invocado. Se le entrega el borrador del Agente_Plato y el Research_Log.md. Ejecuta el algoritmo de auditoría (Sección 5.5).
 - **Output Final:** El CONTROL entrega al usuario el Borrador Final Aprobado y la Tabla de Trazabilidad de Fuentes.

5.0 Frameworks de Razonamiento Explícitos (Algoritmos de Agente)

Esta sección es la especificación técnica *exacta* de lo que cada subagente debe hacer.

5.1. framework_hipotesis.md (Para Agente_Hipotesis, Paso 6)

- **QUÉ (Misión):** Formular una hipótesis (cuantitativa) o un supuesto de trabajo (cualitativo) que sea claro, defendible y comprobable.

- **POR QUÉ (Rationale):** Para asegurar que la investigación sea metodológicamente sólida y no una "pregunta no comprobable".
- **CÓMO (Algoritmo Explícito):**
 1. **Cargar Framework:** `references/10_hipotesis-y-recursos-retoricos.md`.
 2. **Ejecutar "Proceso de Formulación de 5 Pasos":** (1. Pregunta [del Log], 2. Investigación Preliminar [del Log], 3. Borrador, 4. Perfeccionar, 5. Nula) (basado en el archivo fuente).
 3. **Auto-Auditoría (Checklist):** El borrador de hipótesis DEBE ser validado internamente contra el "**Checklist de Calidad de 8 Puntos**" (1. Comprobable, 2. Variables Claras, 3. Lenguaje Claro, 4. Relevante, 5. Específica, 6. Basada en Literatura, 7. Objetiva, 8. Definición Operacional) (basado en el archivo fuente).
 4. **Clasificar:** El *output* DEBE ser clasificado según los "**8 Tipos de Hipótesis**" (ej. "Esta es una H₁ Causal y Direccional") (basado en el archivo fuente).

5.2. `framework_analisis.md` (Para Agente `_HERMENEUTA`, Paso 10 - Cualitativo)

- **QUÉ (Misión):** Analizar el corpus de texto del usuario (PDFs, TXTs de entrevistas) para encontrar patrones de significado.
- **POR QUÉ (Rationale):** Para transformar datos no estructurados (texto) en hallazgos estructurados (temas, códigos) listos para la redacción.
- **CÓMO (Algoritmo Explícito):**
 1. **Cargar Corpus:** Ingestar todos los archivos de texto/PDF proporcionados por el usuario.
 2. **Cargar Frameworks de Análisis:** Cargar los archivos de la base de conocimiento `Skills-para-Escritura-y-Razonamiento-LLM` sobre análisis cualitativo (ej. `Analyzing Scholarly Articles.txt`, `Rhetorical Analysis.txt`, `Critical Reflection.txt`).
 3. **Ejecutar Análisis Temático:**
 - **Paso A (Codificación Abierta):** Leer el material e identificar conceptos iniciales ("códigos").
 - **Paso B (Codificación Axial):** Agrupar los códigos en categorías o "temas".
 - **Paso C (Codificación Selectiva):** Identificar el "tema central" que conecta todas las categorías.
 4. **Verificar Saturación:** Monitorear la redundancia de códigos para identificar la "Saturación Teórica" (el punto donde no surgen nuevos temas).
- **Entregable (Output):** Un (1) **Informe Temático** (.md) que describe los 3-5 temas centrales, con definiciones y citas de evidencia (con localizador de página/párrafo).

5.3. `framework_analisis.md` (Para Agente `_ENGINEER`, Paso 10 - Cuantitativo)

- **QUÉ (Misión):** Analizar datos estructurados (CSVs, Excel) proporcionados por el usuario.
- **POR QUÉ (Rationale):** Para proveer los resultados numéricos (tablas, gráficos) necesarios para probar la hipótesis cuantitativa.
- **CÓMO (Algoritmo Explícito):**
 1. **Cargar Datos:** Ingestar los archivos `.csv` o `.xlsx` del usuario.
 2. **Cargar Hipótesis:** Leer la hipótesis H₁ del `Research_Log` (Paso 6).
 3. **Seleccionar Prueba Estadística:**
 - Si H₁ es Correlacional ("a mayor X, mayor Y"): Seleccionar Análisis de Correlación (ej. `scipy.stats.pearsonr`).
 - Si H₁ es Causal ("X causa Y"): Seleccionar Análisis de Regresión (ej. `sklearn.linear_model.LinearRegression`).
 - Si H₁ es de Diferencia ("Grupo A es diferente de B"): Seleccionar Prueba T o ANOVA (ej. `scipy.stats.ttest_ind`).
 4. **Ejecutar Análisis:** Invocar la capacidad de ejecución de código (Python/R) en un *sandbox* seguro.
- **Entregable (Output):** (1) **Tablas de resultados** (ej. .md con la tabla de regresión y valores *p*), (2) **Gráficos** (ej. .png del gráfico de dispersión), (3) **Informe Descriptivo** (un .md que explica los resultados estadísticos en lenguaje natural).

5.4. `framework_redaccion.md` (Para Agente `_Plato`, Paso 11)

- **QUÉ (Misión):** Sintetizar el `Research_Log` (conceptos, hipótesis, hallazgos verificados) en un texto académico coherente.
- **POR QUÉ (Rationale):** Para narrar la historia de la investigación de forma estructurada y pedagógica, basándose solo en los hechos verificados.
- **CÓMO (Algoritmo Explícito):**
 1. **Cargar Estructura Macro:** `references/07_plantilla-tesis.md`. El *output* debe seguir la "**Plantilla de Tesis de 11 Secciones**" (Resumen, Intro, Problematización, etc.).
 2. **Cargar Estructura Micro (Regla de Oro):** `references/09_construccion-parrafos-academicos.md`. Cada párrafo generado debe seguir el framework **TBTW (Topic-Body-Tokens-Wrap)**.
 - **T (Topic):** La afirmación (del `Research_Log`).

- **B (Body):** La explicación (debe usar "Estrategias de Coherencia").
 - **T (Tokens):** La evidencia (LA ÚNICA FUENTE PERMITIDA es el `Research_Log` o el corpus del usuario, con su cita exacta).
 - **W (Wrap):** El cierre/transición.
- 3. **Cargar Voz (Retórica):** `references/10_hipotesis-y-recursos-retoricos.md`. Usar "Recursos Retóricos" (metáforas, analogías) moderadamente (Balance Logos/Pathos).
- **Entregable (Output):** Un borrador de texto (`.md`) listo para auditoría.

5.5. `framework_auditoria.md` (Para Agente `REVIEWER`, Paso 12)

- **QUÉ (Misión):** Auditar el borrador del Agente `Plato` para asegurar rigor, coherencia y cero plagio.
- **POR QUÉ (Rationale):** Es el control de calidad final que garantiza que el *output* es verificable y cumple con la "**Regla Absoluta**" de integridad.
- **CÓMO (Algoritmo Explícito):**
 1. **Cargar Protocolo de Auditoría:** `references/05_apa-7-citacion-rigurosa.md`.
 2. **Auditoría Capa 1: Coherencia Lógica:**
 - ¿La "Conclusión" (Sección 9 de la plantilla) responde *directamente* a la "Hipótesis" (Paso 6)?
 - ¿La "Discusión" (Sección 8) está lógicamente respaldada por los "Resultados" (Sección 7)?
 - ¿Existen "Falacias" (de `knowledge_base/Fallacies.txt`) en la argumentación?
 3. **Auditoría Capa 2: Protocolo Anti-Plagio de 3 Capas:**
 - **Verificar Parafraseo:** ¿Es un parafraseo válido o "Plagio Estructural" (solo palabras cambiadas)?
 - **Verificar Evidencia:** ¿Todas las afirmaciones (Tokens TBTW) tienen una cita?
 - **Verificar Regla Absoluta:** ¿Todas las citas (directas e indirectas) tienen un localizador (página, párrafo, timestamp)?
- **Entregable (Output):**
 1. Un **Informe de Crítica** (JSON: `{veredicto: "APROBADO" | "RECHAZADO", motivos: [...]}`), enviado al Agente `CONTROL`.
 2. Una **Tabla de Trazabilidad de Fuentes** (formato `.csv` o `.md`), que mapea cada afirmación del *paper* a su fuente y página exacta.

6.0 Tareas del Equipo de Desarrollo (Plan de Acción)

Tarea 1: Implementar la "Skill" Maestra (Agente `CONTROL`)

- **Front-End (UI/UX):** El usuario no debe ver este complejo *workflow*. La UI debe guiar al usuario a través de los 12 pasos de forma lineal (ej. un *wizard* o asistente por pasos).
- **Back-End (Orquestación):**
 - Implementar el "Algoritmo de Bucle Metacognitivo" (Sección 3.1) como el orquestador principal.
 - Gestionar el `Research_Log.md` como el estado central de la sesión (lectura/escritura/actualización).
 - Manejar la lógica del bucle de 3 iteraciones y la consulta al usuario en caso de fallo.
- **Desafío:** Gestión de estado (State Management). El `Research_Log` debe ser robusto y fácil de "parsear" por los agentes.

Tarea 2: Implementar la Base de Conocimiento (Batería)

- **Ingesta de Conocimiento:** Convertir *todos* los archivos `.md` y `.txt` del repositorio `Skills-para-Escritura-y-Razonamiento-LLM` y los 4 *frameworks* centrales en la carpeta `references/` de la Skill.
- **Desafío:** Asegurar que los *prompts* de los subagentes (Tarea 3) referencien correctamente los archivos de esta base de conocimiento.

Tarea 3: Implementar los "Frameworks de Subagentes"

- **Acción:** Redactar los *prompts* de sistema explícitos para cada uno de los agentes definidos en la Sección 5.0 (Hipótesis, Hermeneuta, Engineer, Plato, Reviewer).
- **Desafío:** *Prompt Engineering* de alta precisión. Las instrucciones deben ser algorítmicas, no sugerencias. Deben forzar al LLM a seguir *únicamente* los frameworks proporcionados.

Tarea 4: Implementar Entornos de Ejecución (Qual y Quant)

- **Agente_ENGINEER (Cuantitativo):**
 - Se requiere un entorno de ejecución de código (ej. un *sandbox* de Python con `pandas`, `scipy.stats`).
 - El CONTROL debe poder pasar el `.csv` del usuario al *sandbox* y el ENGINEER debe poder escribir archivos de salida (gráficos `.png`, tablas `.md`).
- **Agente_HERMENEUTA (Cualitativo):**
 - Este agente necesita la capacidad de leer un gran corpus de texto (múltiples PDFs, TXTs). El equipo debe evaluar la mejor manera de "alimentar" este corpus al agente (ej. RAG sobre la carpeta de subida del usuario, o pasar los textos completos si la ventana de contexto lo permite).
 - El entregable clave es la exportación al formato `.xlsx` o `.json` del Libro de Códigos.

Tarea 5: Pruebas de Integridad y Casos de Falla

- **Prueba End-to-End:** Ejecutar el *workflow* completo de 12 pasos con un proyecto de prueba.
- **Prueba de Falla (Crítico):** Alimentar al Agente_REVIEWER con un texto que contenga "Plagio Estructural" y confirmar que lo RECHAZA.
- **Prueba de Falla (Hipótesis):** Alimentar al Agente_Hipotesis con un tema vago y verificar que el Agente_Crítico lo RECHAZA por violar el "Checklist de 8 Puntos".

7.0 Apéndice: Fuentes y Referencias del Proyecto

Documentos de Inspiración Teórica (Modelos de Agentes):

1. **The Denario project:** <https://arxiv.org/abs/2510.26887v1>
2. **Context Engineering 2.0:** <https://arxiv.org/abs/2510.26493v1>
3. **Inverse Knowledge Search (SciencePedia):** <https://arxiv.org/abs/2510.26854v1>

Base de Conocimiento Práctica (Algoritmos de Ejecución):

4. **Repositorio Fuente:** <https://github.com/mroaromero/Skills-para-Escritura-y-Razonamiento-LLM>

5. Frameworks Centrales (Aprobados):

*05_apa-7-citacion-rigurosa.md

07_plantilla-tesis.md

09_construccion-parrafos-academicos.md

*10_hipotesis-y-recursos-retoricos.md